INT315

CLUSTER COMPUTING

UNIT I — Introduction to Apache Spark

Limitations of MapReduce · Batch vs. Real-Time · Stream & In-Memory Processing
Features of Spark · Standalone Installation · Spark vs. Hadoop Ecosystem

Session 2026-27

Companion Study Material — Deep Theory, Comic Analogies, Industry Context & Hands-on Practice
Course Outcomes covered: CO1 (Spark vs. Hadoop) · CO5 (Spark workflows for Big Data)

What's Inside This Unit

How to read each module

Every module follows the same five-beat rhythm: Deep Theory & Mechanics → Visual Analogy → Industry Context → Hands-on Commands/Code → Quick Self-Check.

Diagrams are there to be re-drawn by hand once from memory — that's the single best exam-prep habit for this subject.

MODULE 1 — Limitations of MapReduce in Hadoop

Unit I opens exactly where the syllabus says it should: with the problem Spark was built to solve. Before you can appreciate why Spark exists, you need to feel — not just memorise — why **Hadoop MapReduce**, the original Big Data processing engine, became a bottleneck once real production workloads got big, iterative, and time-sensitive.

1.1 Theory & Mechanics — Where the Time Actually Goes

Recall from INT312 / Module 2 of the Hadoop refresher that a MapReduce job runs as a strict two-phase pipeline: a **Map** phase reads InputSplits from HDFS and emits intermediate key-value pairs, and a **Reduce** phase consumes those pairs after a heavyweight **shuffle-and-sort** step groups them by key across the network. The catch is what happens *between* every one of those phases.

- **Disk-bound intermediate storage.** Every Map task writes its output to LOCAL DISK before the shuffle can even begin, and every Reducer reads that data back over the network before it can start reducing. A single MapReduce job therefore performs at minimum: one HDFS read, one local disk write (map output), one network shuffle read, one local disk write (reduce input spill, if data doesn't fit in the reduce buffer), and one HDFS write (final output). None of that intermediate data ever lives in RAM across phases by design.
- **High latency, especially for multi-stage pipelines.** Real analytics is rarely one MapReduce job — it's a CHAIN: filter, then aggregate, then join, then rank. Classic Hadoop MapReduce has no way to hand data from Job 1 to Job 2 except by writing Job 1's output back to HDFS and having Job 2 read it in again from scratch. Every extra stage in your pipeline re-pays the full disk-and-network tax.
- **Verbosity of code.** A plain word-count in the classic Java MapReduce API needs a Mapper class, a Reducer class, a Driver/main class wiring them together, and explicit type declarations for every key-value pair (Text, IntWritable, LongWritable, ...) — commonly 50-90 lines for a task a modern Spark/PySpark one-liner expresses in 3-4 lines using map, filter, and reduceByKey directly.
- **Poor fit for iterative and interactive workloads.** Machine learning algorithms (e.g. gradient descent) and graph algorithms (e.g. PageRank) reuse the SAME dataset across many rounds. In MapReduce, "many rounds" literally means "many separate jobs, each re-reading the dataset from HDFS from a cold start" — there is no built-in concept of keeping a working set resident in memory between rounds.

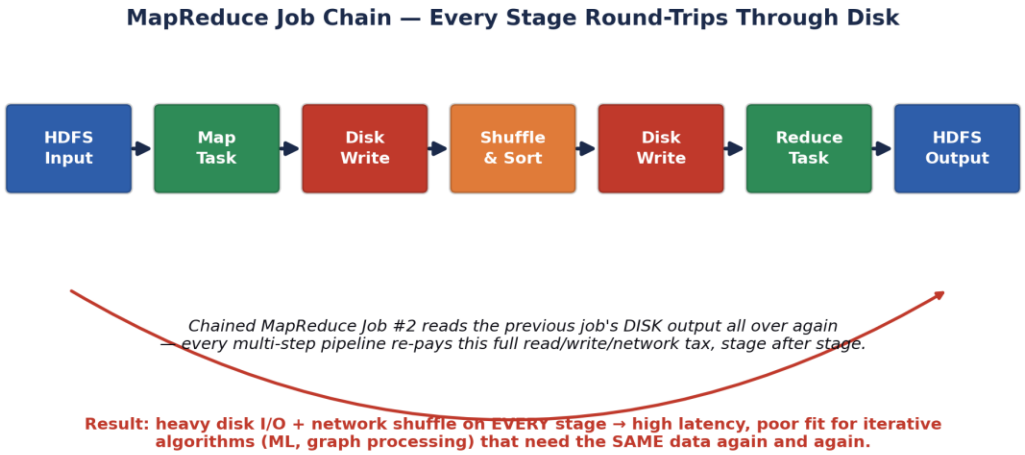


Fig. 1.1 — A chained MapReduce pipeline round-trips through disk at every single stage.

The Courier Who Files a Report After Every Single Step

Imagine a courier (MapReduce) tasked with carrying a message across a 10-department office. At EVERY department, company policy forces the courier to stop, print the message on paper, file a physical copy in the department's cabinet, and only then walk it — freshly reprinted — to the next department.

The courier is diligent and never loses a message (that's fault tolerance you actually want to keep). But if the same 10-department round trip has to happen 50 times because the CEO keeps asking follow-up questions (iterative ML training), that's 50 x 10 = 500 filing-cabinet trips for information that barely changed.

Spark's answer, coming up in Module 3, is basically: let the courier hold the message in their HAND across departments and only file a physical copy when someone actually asks for a permanent record.

Why production teams cared enough to replace this

This wasn't an academic complaint — it showed up directly on the invoice. Iterative ML jobs and multi-stage ETL pipelines running on classic MapReduce could take HOURS where the equivalent Spark job, given enough cluster RAM, finished in MINUTES, because every retry of a stage was a full disk round trip rather than a RAM lookup.

Cluster time is billed by the hour (on-prem depreciation or cloud instance-hours either way), so a 10x latency gap is a direct, measurable cost difference — this is the single most common opening answer to "why did your company migrate off MapReduce?" in interviews.

1.2 Quick Check — Module 1

Self-Assessment

1) Name the two separate points in a single MapReduce job where data is forced to disk, even before you chain a second job onto it.

2) Why does chaining three MapReduce jobs (filter → aggregate → rank) cost roughly 3x the HDFS I/O of running the equivalent as one Spark pipeline?

3) Hands-on: Write pseudocode (Java-MapReduce-style class stubs are fine) for a word count job, then count the lines. Compare it mentally against the PySpark one-liner sc.textFile(f).flatMap(...).map(...).reduceByKey(...) you'll write in Module 4 — note the ratio.

MODULE 2 — Batch vs. Real-Time Analytics

Once you understand WHY MapReduce is slow, the next natural question is: slow *compared to what requirement?* That requirement is latency — and Big Data systems are usually classified by how much latency between "an event happens" and "an answer is available" they're built to tolerate.

2.1 Definitions & the Latency Trade-off

- **Batch analytics —** processes a large, bounded (finite, already-collected) dataset all at once, on a schedule (hourly, nightly, weekly). Latency is measured in minutes to hours. Classic MapReduce and most traditional Spark jobs (spark-submit reading a fixed HDFS directory) are batch by default. Optimised for THROUGHPUT — process as much data as cheaply as possible per unit time — not for speed-of-first-answer.
- **Real-time (stream) analytics —** processes an unbounded, continuously-arriving stream of events as they happen, with latency measured in milliseconds to a few seconds. Spark Streaming / Structured Streaming, Kafka Streams, and Flink live here. Optimised for LOW LATENCY on small, continuous increments of data rather than raw throughput on one giant batch.
- **Micro-batch —** the middle ground Spark Streaming actually uses under the hood: it slices the incoming stream into very small time-boxed batches (e.g. every 1-2 seconds) and runs a tiny batch job on each slice. It feels real-time to a human but is architecturally still "lots of small batch jobs," which is why Module 5's Spark Streaming unit will feel familiar once you know RDDs.

2.2 Lambda vs. Kappa Architecture — Two Ways to Get Both

In practice, most companies need BOTH an accurate historical view and a fresh, up-to-the-second view. Two competing architectural patterns solve this:

- **Lambda Architecture —** runs a Batch Layer (recomputes accurate results over ALL historical data, e.g. nightly) and a Speed Layer (a stream processor patching in only the last few minutes of approximate results) side by side, merging both in a Serving Layer that queries stitch together. Pro: the batch layer's periodic full recompute self-heals any bugs in the speed layer. Con: you now maintain TWO codebases (batch logic and streaming logic) that must stay logically equivalent — a well-known maintenance headache.
- **Kappa Architecture —** treats EVERYTHING as a stream, including "batch," which becomes just "replay the log from the beginning." One codebase (a stream processor like Spark Structured Streaming reading from Kafka) serves both fresh and (via replay) historical results. Pro: one codebase to maintain. Con: replaying a very long history through a stream processor can itself be slow, and it depends on your message log (Kafka) retaining history long enough to replay.

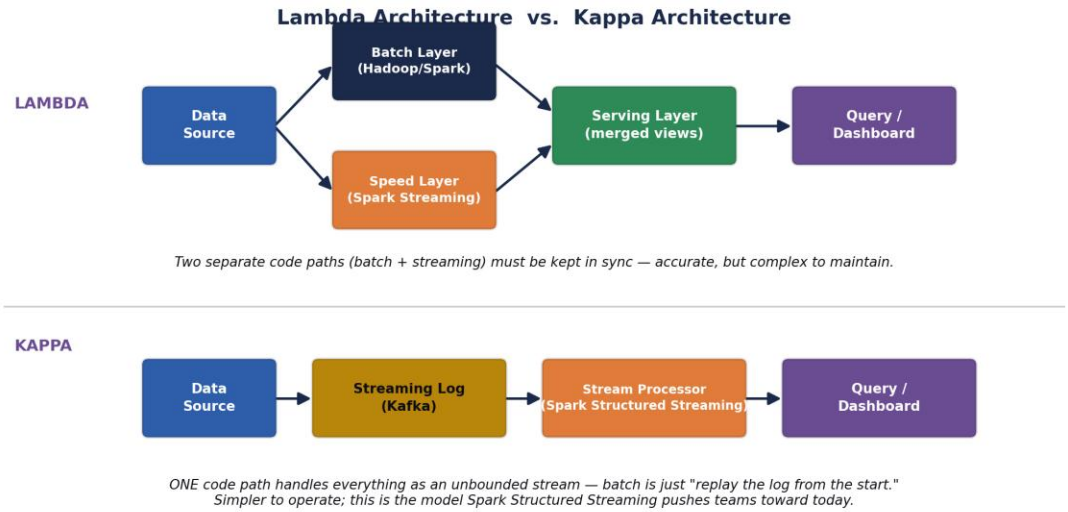


Fig. 2.1 — Lambda keeps two parallel pipelines in sync; Kappa collapses everything into one streaming pipeline.

Two Restaurants Preparing the Same Dish

Lambda Architecture is a restaurant with two kitchens: a slow-cooking kitchen (batch layer) that spends all night making a perfect, deeply-flavoured stew from every ingredient delivered that week, and a fast wok-station (speed layer) that can throw together an approximate version of tonight's dish in two minutes for a hungry walk-in customer. The head waiter (serving layer) blends both onto the plate so the customer gets "mostly the good stew, freshened up with tonight's ingredients."

Kappa Architecture fires the slow kitchen entirely. There's just ONE wok-station, and if a customer wants "the full week's stew," the chef simply reruns every single ingredient delivery ticket from the start of the week through the SAME wok, one at a time, in order. Simpler staffing, but that "replay the whole week" trick only works if the ingredient delivery log itself was kept.

Where each pattern shows up in the wild

Fraud detection at a payments company: Kappa is common — every transaction is a stream event, and a "batch report" is genuinely just replaying yesterday's transaction stream through the same fraud-scoring logic used live.

Retail analytics dashboards: Lambda is still common where the nightly batch job runs complex joins across many warehouse tables that would be awkward to express as a pure stream, while a lightweight speed layer just patches in "orders in the last hour."

2.3 Quick Check — Module 2

Self-Assessment

1) A hospital dashboard needs to show a patient's heart-rate alert within 2 seconds of an anomaly. Is this a batch or real-time analytics requirement, and why does that rule out classic MapReduce entirely?

2) What single operational cost does Kappa Architecture save you compared to Lambda, and what does it demand from your message log (e.g. Kafka) in exchange?

3) Hands-on: Sketch (on paper) a Lambda Architecture for a ride-hailing app that needs both "driver's live location" (real-time) and "monthly earnings report" (batch). Label which box is the batch layer and which is the speed layer.

MODULE 3 — Applications of Stream Processing & In-Memory Processing

3.1 Theory & Mechanics — What Changes When RAM Replaces Disk

This is the architectural idea that makes everything else in this course possible, so it's worth being precise: Spark's core innovation is the **Resilient Distributed Dataset (RDD)** — a read-only, partitioned collection of data that CAN be kept resident in executor RAM across multiple operations, instead of being written back to disk after every single transformation. You'll build RDDs by hand in Unit III; for now, focus on the consequence, not the API.

- **In-memory computing, precisely defined.** RAM read/write latency is roughly 100,000x faster than a spinning disk seek and still 10-20x faster than even a fast SSD for random access patterns. When a Spark job calls .cache() or .persist() on an RDD/DataFrame, Spark keeps that data's partitions in executor memory (spilling to disk only if it doesn't fit), so every SUBSEQUENT action against that same data reads from RAM instead of re-computing or re-reading from HDFS.
- **Stream processing, precisely defined.** A stream processor keeps a small amount of STATE (e.g. "running total per user in the last 5 minutes") resident in memory and updates it incrementally as each new event arrives, rather than re-scanning a whole historical dataset for every new event. This is only practical because that state fits, and stays, in RAM.

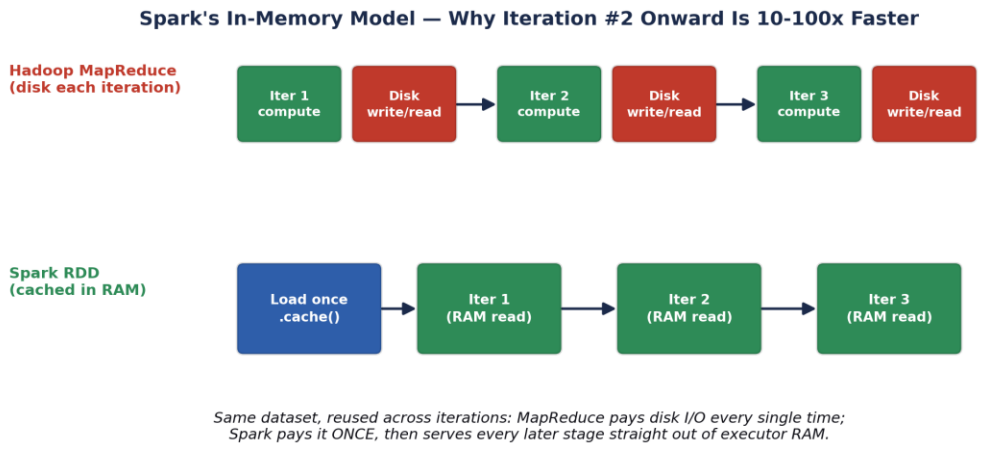


Fig. 3.1 — Caching a dataset once in RAM turns every later iteration into a RAM read instead of a disk round-trip.

3.2 Real-World Use Cases

- **Fraud detection.** A card-payments platform scores every transaction against a running per-account, per-merchant-category profile held in memory (average spend, typical location, typical hour). Because that profile lives in RAM and updates incrementally, a fraud score can be returned in single-digit milliseconds — batch MapReduce, re-scanning transaction history from disk, simply cannot hit that latency budget.

- **Real-time dashboards.** Operations dashboards (e.g. live order volume, live server error rate) aggregate a continuous event stream into rolling windows ("errors in the last 60 seconds") held in memory, refreshing the visible number every few seconds instead of waiting for tomorrow's batch report.
- **Iterative machine learning.** Training algorithms like logistic regression or K-means repeatedly scan the SAME training set to update model weights each round. Caching that training RDD once in memory (Unit III) is precisely why Spark MLlib (Unit VI) is practical at scale where hand-rolled MapReduce ML rarely was.
- **Session analytics / recommendation engines.** An e-commerce site keeping a user's current browsing session (last N products viewed) in an in-memory structure can serve "you might also like" recommendations before the next page even loads.

The Chef With a Prep Station vs. The Chef Who Walks to the Warehouse

Picture two chefs cooking the same 200-dish tasting menu. Chef Disk walks out to the warehouse (disk) to fetch each ingredient fresh, every single time a recipe calls for it — even if the SAME onion was already fetched for the previous three dishes.

Chef RAM instead preps a station: chops the onions ONCE, sets them in a bowl right next to the stove (cached in memory), and every dish that needs onions just reaches into the bowl. Chef RAM finishes the tasting menu in a fraction of the time, at the cost of needing enough counter space (RAM capacity) to hold the prepped bowl.

⚠ The catch — this isn't free

RAM is far more expensive per GB than disk, and it's volatile: an executor crash loses whatever wasn't checkpointed. Spark's RDD lineage graph (Unit III) is precisely how Spark recovers cached data cheaply after a crash — by RECOMPUTING only the lost partitions from their recorded lineage, not by treating the cache as a single point of failure.

3.3 Quick Check — Module 3

Self-Assessment

1) Explain, in your own words, why caching an RDD in RAM does NOT remove the need for fault tolerance — what actually happens if a cached partition is lost mid-job?

2) A dashboard needs "error count in the last 60 seconds, updated every 5 seconds." Would you reach for a batch job or a stream processor, and what has to live in memory for it to work?

3) Hands-on: List 3 applications from your own daily life (apps you use) that are almost certainly backed by real-time stream processing rather than nightly batch jobs, and justify each in one sentence.

MODULE 4 — Features & Benefits of Apache Spark

4.1 Speed

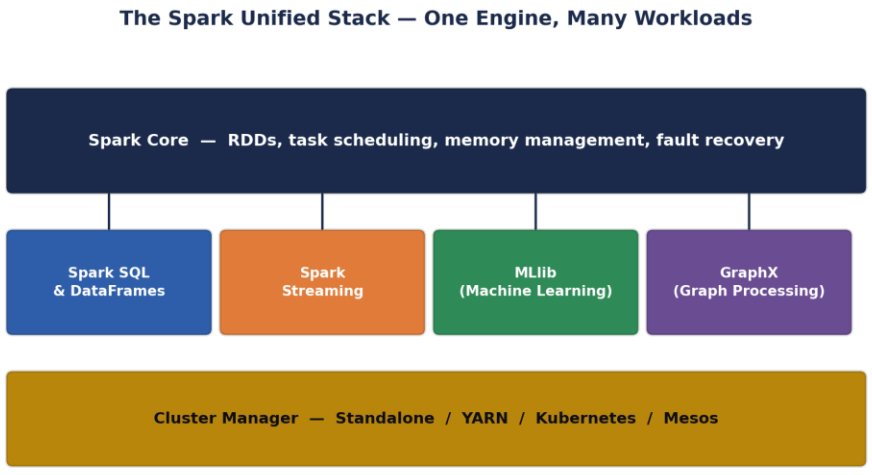
Speed in Spark comes from two independent sources that are easy to conflate: (1) **in-memory caching** (Module 3) avoiding repeated disk I/O across iterations, and (2) the **DAG execution engine** (below) building an optimised physical plan across an ENTIRE chain of transformations before running anything, rather than executing each MapReduce job as an isolated, unoptimised unit. Independent benchmarks commonly cite 10-100x speedups for iterative workloads and lower but still substantial gains even for pure disk-to-disk batch jobs, purely from better DAG-level scheduling and reduced job-startup overhead.

4.2 Ease of Use

- **Rich, expressive APIs.** Spark exposes the SAME core operations (map, filter, join, groupBy, reduce) consistently across Scala, Python (PySpark), Java, R, and SQL — you'll write real code in Scala starting Unit II and in PySpark starting Unit VI.
- **Interactive shells.** spark-shell (Scala) and pyspark (Python) let you explore a dataset line-by-line and see results immediately — something classic MapReduce, which requires a full compile-package-submit cycle per attempt, never offered.
- **High-level structured APIs.** DataFrames and Spark SQL (Unit IV) let you express many transformations as declarative SQL-like operations instead of hand-written low-level key-value logic.

4.3 Unification — One Engine, Many Workloads

This is arguably Spark's single biggest architectural win over the classic Hadoop world: instead of stitching together separate specialised tools (MapReduce for batch, a different streaming framework, a different ML framework, a different graph framework — each with its own cluster deployment, its own failure model, its own data-copy overhead between tools), Spark runs SQL, Streaming, ML, and Graph workloads on the SAME Spark Core engine and the SAME RDD/DataFrame abstraction.



SQL, Streaming, ML and Graph code all compile down to the SAME RDD/DAG engine underneath.

Fig. 4.1 — Spark SQL, Streaming, MLlib and GraphX are libraries on top of one shared Core engine and cluster manager.

4.4 Lazy Evaluation

Spark operations split into two categories: **transformations** (map, filter, join — return a NEW RDD/DataFrame describing what should happen) and **actions** (collect, count, save — actually trigger computation and return/persist a result). Transformations are LAZY: calling .map() does not touch a single row of real data — it just adds a node to a logical plan (a DAG). Only when an action is called does Spark's Catalyst/DAG scheduler walk that plan backwards, optimise it (e.g. fusing a map and a filter into one pass, or skipping columns nobody asked for), and finally execute.

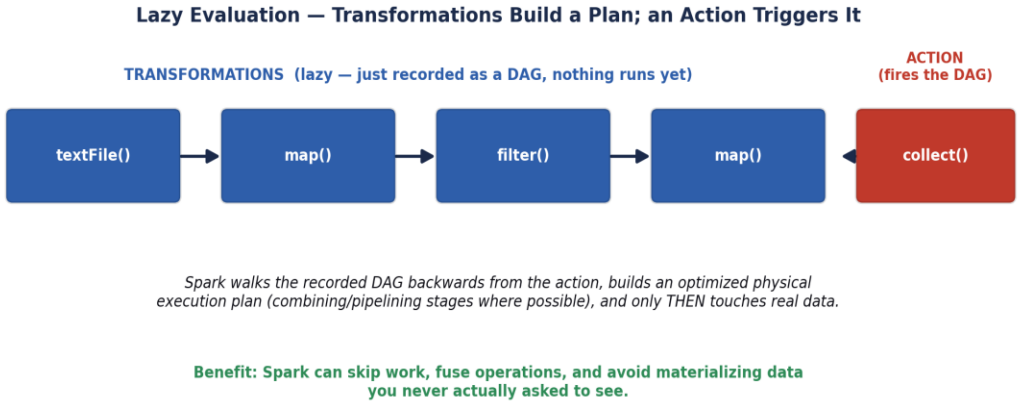


Fig. 4.2 — Transformations only build a plan; nothing executes until an action fires it.

The Architect Who Never Lifts a Brick Until You Sign the Final Plan

Imagine hiring an architect (Spark) and describing changes one at a time: "add a window here," "move that wall," "knock down this doorway." An EAGER architect would run out and physically make each change the moment you say it — including undoing work if your next instruction contradicts the last.

Spark's architect instead just keeps redrawing the BLUEPRINT (the DAG) in response to every instruction, touching zero bricks. Only when you say "build it" (call an action like .collect() or .save()) does the architect look at the FINAL blueprint as a whole, spot redundant steps ("you asked for a window here then a wall right over it — skip the window"), and execute the optimised plan in one efficient pass.

Why this matters beyond "it's a nice trick"

Lazy evaluation is what lets Spark's Catalyst optimizer (Unit IV, Spark SQL) automatically push filters down before expensive joins, skip reading columns your query never references (column pruning), and avoid materialising huge intermediate results you never asked to see — this is a large, measurable chunk of WHY Spark SQL often outperforms even hand-tuned procedural code doing the same logical work.

4.5 Quick Check — Module 4

Self-Assessment

1) List the four workload libraries that sit on top of Spark Core, and name the ONE thing they all ultimately compile down to.

2) Is df.filter(col('age') > 30) a transformation or an action? What actually happens on the cluster the moment that line executes, and what happens later when you call df.show()?

3) Hands-on: Open a Python shell (even without Spark installed yet) and write out, as comments, which lines in a 5-step pandas-style pipeline you'd expect to be "lazy" if it were Spark instead — mark each line T (transformation) or A (action).

MODULE 5 — Installation of Spark as a Standalone User

"Standalone" here has a specific, exam-relevant meaning: it refers to Spark's OWN built-in, lightweight cluster manager (as opposed to running Spark ON TOP OF YARN or Kubernetes). Standalone mode is the fastest way to get a real, working multi-process Spark cluster running on your own laptop for learning — a single Master daemon plus one or more Worker daemons, all launched from plain shell scripts with zero external dependencies beyond Java and Spark itself.

5.1 Prerequisites Check

- Java 8, 11, or 17 (a JDK, not just a JRE) — Spark runs on the JVM.
- Python 3.8+ if you plan to also use PySpark later in Unit VI.
- At least 4 GB free RAM for a comfortable local single-machine "cluster."

5.2 Step-by-Step Installation (Linux / POSIX)

Step 1 — Verify / install Java

```
# Check if Java is already installed
java -version

# If missing, on Debian/Ubuntu:
sudo apt update
sudo apt install -y openjdk-17-jdk
```

Step 2 — Download and extract Spark

```
# Download the pre-built-for-Hadoop Spark release
cd ~
wget https://downloads.apache.org/spark/spark-3.5.1/spark-3.5.1-bin-hadoop3.tgz

# Extract it into /opt (conventional home for manually installed software)
sudo tar -xzf spark-3.5.1-bin-hadoop3.tgz -C /opt/
sudo mv /opt/spark-3.5.1-bin-hadoop3 /opt/spark
```

Step 3 — Set environment variables

```
# Append to ~/.bashrc (or ~/.zshrc)
echo 'export SPARK_HOME=/opt/spark' >> ~/.bashrc
echo 'export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin' >> ~/.bashrc
echo 'export JAVA_HOME=$(dirname $(dirname $(readlink -f $(which java))))' >> ~/.bashrc

# Reload the shell config
```

```
source ~/.bashrc
```

Step 4 — Start the Master and Worker daemons

```
# Start the Master daemon (Web UI defaults to port 8080)
$SPARK_HOME/sbin/start-master.sh

# Start ONE Worker and register it with the Master
# spark://<hostname>:7077 is printed in the Master's log / Web UI
$SPARK_HOME/sbin/start-worker.sh spark://localhost:7077
```

Step 5 — Verify with the interactive shell

```
# Launch the Scala interactive shell against your standalone cluster
$SPARK_HOME/bin/spark-shell --master spark://localhost:7077

# Inside the shell, run a trivial sanity-check job:
scala> val nums = sc.parallelize(1 to 1000000)
scala> nums.filter(_ % 7 == 0).count()
```

Step 6 — Stop the cluster when done

```
$SPARK_HOME/sbin/stop-worker.sh
$SPARK_HOME/sbin/stop-master.sh
```

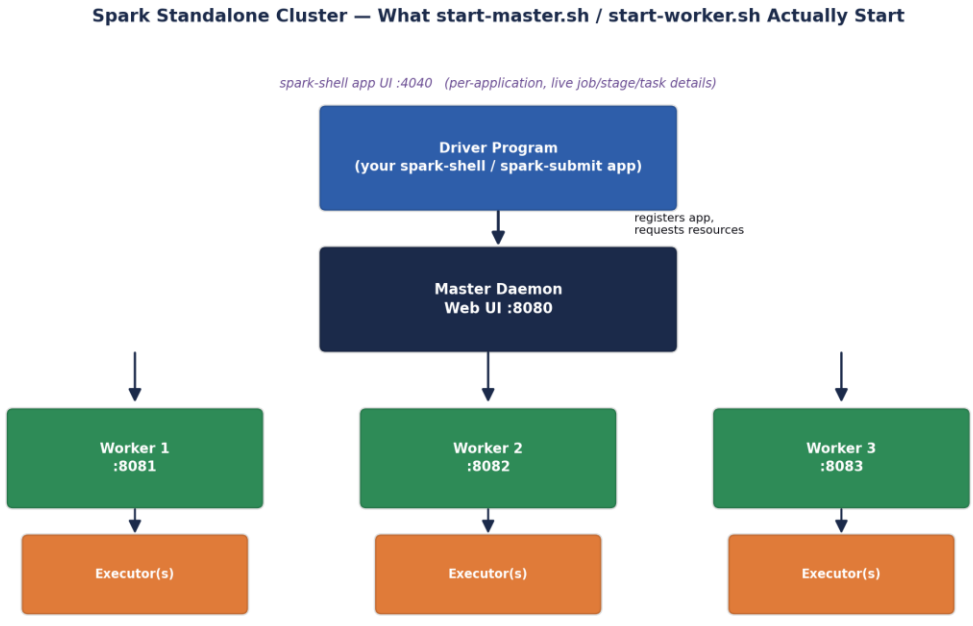


Fig. 5.1 — start-master.sh and start-worker.sh launch the same processes your spark-shell session talks to.

5.3 Verifying the Web UIs

Port	What it shows	When it's available
8080	Master Web UI — registered workers, running & completed applications, cluster-wide resource usage.	As soon as start-master.sh runs, for the life of the cluster.
8081 (+1 per worker)	Worker Web UI — this worker's executors and resource usage.	As soon as start-worker.sh runs on that machine.
4040	Application (Driver) UI — live jobs, stages, tasks, DAG visualisation, storage (cached RDDs), SQL query plans.	Only while a spark-shell / spark-submit application is actively running.

⚠ The #1 beginner installation mistake

The port-4040 Application UI disappears the instant your spark-shell session or job exits — it is NOT a persistent dashboard like port 8080. If a screenshot or grade-check requires port 4040, take it WHILE spark-shell is still open, not after.

5.4 Quick Check — Module 5

Self-Assessment

1) Which environment variable tells spark-shell where to find a working JVM, and which tells it where Spark itself was extracted?

2) You run start-worker.sh but the Master's Web UI (port 8080) never shows the new worker. Name two likely causes (hint: think about the spark://host:port string and firewall/network reachability).

3) Hands-on: On your own machine (or a cloud shell), actually run Steps 1-5 above, and take a screenshot of the port 8080 Master UI showing at least one registered Worker.

MODULE 6 — Compare Spark vs. Hadoop Ecosystem

This closing module of Unit I directly targets **CO1** — "analyze the similarities and differences between Spark and Hadoop." The single most accurate, interview-safe framing, worth memorising verbatim: Spark primarily replaces the **MapReduce PROCESSING ENGINE** layer — it commonly still runs ON TOP OF HDFS for storage and can still be launched BY YARN for resource management. "Spark replaced Hadoop" is imprecise; "Spark replaced MapReduce" is the precise claim.

6.1 Direct Comparison Matrix

Dimension	Hadoop (MapReduce)	Apache Spark
Storage	HDFS (also usable by Spark)	No storage of its own — reads HDFS, S3, local FS, HBase, Kafka, etc.
Compute model	Disk-based, two-phase Map/Reduce per job	In-memory RDD/DataFrame DAG, chainable transformations
Speed (iterative jobs)	Baseline — full disk round-trip every round	Commonly 10-100x faster once data is cached in RAM
Ease of use / API	Verbose Java Mapper/Reducer classes	Concise Scala/Python/SQL/R APIs, interactive shell
Real-time processing	Not supported natively (batch only)	Native support via Spark Streaming / Structured Streaming
Fault tolerance	Replicated HDFS blocks + task re-execution	RDD lineage graph — lost partitions recomputed from lineage
Resource management	YARN (built alongside Hadoop 2.x)	Can use its OWN Standalone manager, or YARN, or Kubernetes, or Mesos
Ecosystem breadth	MapReduce + Hive + HBase + Pig + Sqoop + Oozie + ZooKeeper	One engine: Spark Core + SQL + Streaming + MLlib + GraphX
Typical hardware cost	Cheaper — commodity disks are the main capacity driver	Costlier per node — needs enough RAM to make caching worthwhile
Best fit today	Very large one-pass batch ETL where cost-per-TB dominates	Iterative ML, interactive analytics, streaming, mixed SQL+ML+Graph pipelines

6.2 What Spark Deliberately Reuses From Hadoop

- HDFS — Spark commonly reads its input FROM and writes its output TO HDFS; it did not reinvent distributed storage.
- YARN — a Spark job can be submitted with --master yarn, letting YARN's ResourceManager schedule Spark executors exactly like it schedules MapReduce tasks, on the SAME shared cluster.
- Hive Metastore — Spark SQL can read table definitions straight out of an existing Hive Metastore, so migrating a Hive-based warehouse to Spark SQL doesn't require redefining every table.

Two Postal Systems Sharing the Same Roads

Picture Hadoop MapReduce as the original national postal service: reliable, government-run, delivers everything eventually, but every single parcel is physically re-weighed, re-stamped, and re-filed at every sorting depot along the route (disk round-trips).

Spark is a newer private courier that DOES NOT build its own roads or warehouses — it drives on the exact same national highway system (HDFS) and can even use the same regional dispatch centers (YARN) — but its trucks carry parcels straight through in one continuous trip whenever possible, only stopping at a depot when a parcel genuinely needs to be archived (an action like .save()).

Interviewers love this analogy because it forces the precise answer: Spark didn't tear down the roads (HDFS) or the dispatch centers (YARN) — it replaced the SLOW COURIER (MapReduce) that used to drive on them.

6.3 Quick Check — Module 6

Self-Assessment

1) True or false, with justification: "Migrating to Spark means you no longer need HDFS."

2) Name one Hadoop-ecosystem component Spark commonly runs ON TOP OF for (a) storage and (b) resource management.

3) Hands-on: Fill in your own one-row addition to the comparison matrix above for the dimension "Programming language support" — research and write both cells yourself.

UNIT I RECAP — Course Outcome Mapping

Per the official INT315 syllabus, Unit I is where two Course Outcomes first get exercised. Mapping your Unit I learning explicitly onto them now makes end-of-semester revision far faster:

CO	Outcome (as written in the syllabus)	Where Unit I builds it
CO1	Analyze the similarities and differences between Spark and Hadoop.	Modules 1 & 6 — MapReduce's disk-bound design, and the full head-to-head comparison matrix.
CO5	Analyze workflows as well as processing and analysis of Big Data using Apache Spark.	Modules 2-5 — batch vs. real-time workflow design, in-memory processing model, Spark's unified stack, and hands-on cluster setup.

Big-Picture Summary

- MapReduce is disk-bound and verbose — every stage round-trips through disk, and multi-stage pipelines pay that tax repeatedly.
- Batch trades latency for throughput/accuracy; real-time (stream) trades some accuracy/complexity for millisecond-scale latency; Lambda and Kappa are the two dominant patterns for getting both.
- Spark's core trick is keeping reusable data in executor RAM (caching), which is what unlocks fast iterative ML, graph processing, and low-latency streaming.
- Spark is fast (in-memory + optimised DAG execution), easy to use (concise multi-language APIs, interactive shell), and unified (SQL + Streaming + ML + Graph on one engine) — all enabled by lazy evaluation.
- Standalone mode is Spark's own lightweight cluster manager: start-master.sh + start-worker.sh, verified via the Master UI (:8080), Worker UI (:8081+), and the live per-application UI (:4040).
- Spark did not replace Hadoop wholesale — it replaced the MapReduce PROCESSING ENGINE, while commonly still running on HDFS storage and/or under YARN resource management.

UNIT I REVIEW — Quiz & Hands-on Capstone Exercise

Three-Question Review Quiz

Q1 — Conceptual

A team's nightly Hadoop MapReduce ETL pipeline chains four separate jobs (clean → dedupe → join → aggregate) and takes 3 hours. They rewrite it as a single Spark pipeline with intermediate DataFrames cached in memory, and it now finishes in 20 minutes.

Explain, using the specific architectural terms from Module 1 and Module 3 (not just "Spark is faster"), WHY chaining four MapReduce jobs was so much slower than the equivalent cached Spark pipeline.

Q2 — Applied

You are designing a system for a food-delivery app that must (a) show a live "driver ETA" updating every few seconds, AND (b) generate an accurate monthly revenue-by-city report.

Would you reach for a pure batch design, a pure streaming design, or a Lambda/Kappa hybrid? Justify your choice and name which Spark component (from Module 4's unified stack) handles each half of your answer.

Q3 — Comparison

A colleague says: "We're moving off Hadoop entirely — no more HDFS, no more YARN, just Spark." Based on Module 6, is this colleague describing a common, realistic migration path? Rewrite their sentence to be architecturally precise.

Hands-on Capstone Exercise

Build & prove your own Standalone cluster

1) Complete the full Module 5 installation on your own machine (or a cloud shell/VM) — Java check through Step 6.

2) Start the Master and ONE Worker, and take a screenshot of the port 8080 Master UI showing the registered worker with its core/memory allocation visible.

3) Launch spark-shell against your standalone master (spark://localhost:7077) and run: sc.parallelize(1 to 10000000).filter(_ % 3 == 0).count() — record how long it takes and the result.

4) While that shell session is still open, screenshot the port 4040 Application UI, specifically the Jobs tab showing the job you just ran.

5) Write 3-4 sentences (in your own words) connecting what you SAW on these two dashboards back to the Driver / Master / Worker / Executor architecture diagram from Module 5.

Submission note

Both screenshots (port 8080 AND port 4040) are required — they demonstrate two different, non-overlapping layers of the architecture, and grading checks for both.